

T1: Problemas Clássicos de Concorrência

FPPD - Fundamentos de Processamento Paralelo e Distribuído (98713-04) | **Eduardo Hoffmann e Lucas Mocelin**

Repositório: <https://github.com/theeduardohoffmann/go-concurrency-benchmarks>

Ambiente: Go 1.27, Windows 11, 12 CPUs lógicas. Os dois programas foram executados com `go run -race` sem nenhum aviso de data race. Os dados abaixo estão em `relatorio/dados/` (execuções reais, sem `-race` para não distorcer o tempo). O `time.Sleep` do Windows tem granularidade de cerca de 1 ms, então os tempos valem mais para comparar configurações entre si do que como valores absolutos.

1. Problema 1: Jantar dos Filósofos

1.1 Solução e decisões de projeto

Cada garfo é um `chan struct{}` de capacidade 1: se há um token no channel, o garfo está livre. Pegar o garfo é receber do channel e soltar é enviar de volta. Cada filósofo é uma goroutine que repete: pensar, pegar os garfos, comer, soltar. O programa recebe `-n` (número de filósofos) e `-r` (iterações). Interpretamos `R` como o **número total de refeições da mesa** (um contador atômico decrementado a cada refeição): assim cada filósofo come o quanto conseguir e a distribuição das refeições mostra se a estratégia é justa. Se cada filósofo fizesse `R` refeições, todos terminariam com o mesmo número por construção e não haveria o que comparar.

Versão base: pega a esquerda, espera um pouco (`-segurar`, 2 ms, aplicada a todas as versões) e pega a direita. A pausa faz o deadlock acontecer de forma reproduzível. **Estratégias que eliminam o deadlock:** (1) **hierarquia de recursos:** os garfos são numerados e cada filósofo pega primeiro o de menor número (só o filósofo `N-1` inverte a ordem); (2) **limite de N-1 à mesa:** um channel de capacidade `N-1` funciona como semáforo (a 'sala'): o filósofo entra na sala antes de pegar garfos e sai depois de comer; (3) **garçom:** uma goroutine recebe os pedidos por um channel, em ordem de chegada, retira os dois garfos do filósofo e só então responde; o filósofo come e devolve os garfos.

1.2 Encerramento ordenado

O filósofo termina quando as `R` refeições acabam; a main espera todos com `sync.WaitGroup`. No garçom, depois disso a main fecha o channel de pedidos, o `for` range do garçom acaba e a main espera um channel `garcomFim`. A espera por um garfo é um `select` com `time.After` (1 s): se um filósofo espera tanto assim, o programa conclui que há deadlock, imprime quem segura/espera qual garfo e a goroutine retorna. Assim, até a versão que trava termina sem goroutines bloqueadas.

1.3 Análise de deadlock e starvation

Condições de Coffman: (a) exclusão mútua, (b) posse e espera, (c) não preempção e (d) espera circular. O deadlock exige as quatro; cada estratégia quebra uma.

Versão	Deadlock?	Justificativa (Coffman)	Starvation?
base	Sim: 3 de 3 execuções. Saída: filósofo <code>i</code> esperando o garfo <code>i+1</code> , para todo <code>i</code> (0 espera 1, 1 espera 2, ..., 4 espera 0).	As quatro condições valem: garfo é exclusivo, cada um segura o da esquerda e pede o da direita, ninguém é forçado a soltar e as esperas formam um ciclo.	Sim, como consequência do deadlock: ninguém mais come.
hierarquia	Não (0 deadlocks em 6 execuções: 3 normais e 3 de estresse).	Quebra a espera circular (d): como todos pegam em ordem crescente, um ciclo de espera exigiria uma ordem decrescente em algum ponto, o que não existe.	Não há starvation total, mas é injusta : os filósofos 0 e 4 comem bem menos (ver 1.4).
limite (N-1)	Não (0 em 6 execuções).	Com no máximo <code>N-1</code> filósofos disputando <code>N</code> garfos, sempre sobra um garfo para algum deles; ele come e libera. Quebra a espera circular (d), pois o ciclo completo precisaria de <code>N</code> filósofos.	Não: o channel-semáforo atende em ordem de chegada; refeições quase idênticas.
garçom	Não (0 em 6 execuções).	Quebra a posse e espera (b): só o garçom retira garfos, e o filósofo só recebe quando já tem os dois. Quem segura um garfo está comendo e vai soltá-lo.	Não: fila FIFO; o pedido mais antigo é sempre o próximo, então ninguém é ultrapassado.

A ausência de starvation nas versões corretas depende de dois fatos: a refeição dura um tempo finito e as filas de espera dos channels do Go são servidas em ordem de chegada (comportamento da implementação do runtime).

1.4 Dados de execução e fairness

Estratégia	Refeições por filósofo (F0 F1 F2 F3 F4)	Espera média pelos garfos em ms (F0 ... F4)	Tempo total
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': ' Carga normal (pensa até 2 ms), N=5, R=1000, execução 1' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7.8, greek=0, italic=0, link=[], rise=0, text='Carga normal', textColor=Color(0,0,0,1), us_lines=[]), ParaFrag(__tag__='para', bold=0, fontName='Helvetica', fontSize=7.8, greek=0, italic=0, link=[], rise=0, text=' (pensa até 2 ms), N=5, R=1000, execução 1', textColor=Color(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph			

Estratégia	Refeições por filósofo (F0 F1 F2 F3 F4)	Espera média pelos garfos em ms (F0 ... F4)	Tempo total
base	0 0 0 0 0 (deadlock)	-	1,0 s (timeout)
hierarquia	163 207 218 248 164	4,67 3,13 2,76 2,07 4,69	1,26 s
limite (N-1)	199 199 201 201 200	4,69 4,61 4,64 4,58 4,67	1,54 s
garçom	198 201 200 204 197	1,68 1,70 1,70 1,66 1,79	0,91 s
Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': ' Carga de estresse (não pensa: sempre faminto), N=5, R=1000, execução 1' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=7.8, greek=0, italic=0, link=[], rise=0, text='Carga de estresse', textColor=Color(0,0,0,1), us_lines=[]), ParaFrag(__tag__='para', bold=0, fontName='Helvetica', fontSize=7.8, greek=0, italic=0, link=[], rise=0, text=' (não pensa: sempre faminto), N=5, R=1000, execução 1', textColor=Color(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph			
base	0 0 0 0 0 (deadlock)	-	1,0 s (timeout)
hierarquia	135 193 218 318 136	7,68 4,94 4,23 2,49 7,61	1,24 s
limite (N-1)	200 200 200 200 200	6,20 6,13 6,25 6,15 6,25	1,55 s
garçom	200 200 200 200 200	6,81 6,68 6,77 6,77 6,76	1,69 s

Cada configuração foi executada 3 vezes e o padrão se repetiu (hierarquia: de 157 a 163 refeições do filósofo 0 e de 248 a 257 do filósofo 3 na carga normal; de 131 a 137 contra 316 a 321 sob estresse). A tabela mostra a execução 1. Nas execuções 1 e 2 da base na carga normal ninguém chegou a comer e na 3 foram só 2 refeições.

Hierarquia evita o deadlock mas é a estratégia menos justa. Sob estresse, os filósofos 0 e 4 comem cerca de 135 vezes, contra cerca de 318 do filósofo 3, e esperam 3 vezes mais. Nossa hipótese (não testada separadamente): os filósofos 0 e 4 pegam ambos primeiro o garfo 0 e o seguram enquanto esperam o segundo garfo, atrapalhando-se mutuamente, enquanto o filósofo 3 tem menos concorrência. **Limite (N-1)** é justo (200 refeições para todos sob estresse) porque a fila da sala é FIFO. **Garçom** também é justo. Sobre desempenho, o garçom foi o mais rápido na carga normal (~0,9-1,0 s contra ~1,4-1,5 s do limite e ~1,2 s da hierarquia), mas sob estresse ficou o mais lento (~1,6-1,7 s): como atende um pedido de cada vez em ordem estrita, o garçom pode ficar esperando um garfo ocupado enquanto outro pedido atrás dele poderia ser atendido; quando os filósofos pensam, essa fila quase não se forma. Concluímos que hierarquia é a mais simples, mas a menos justa; limite e garçom são justas, com custos de desempenho diferentes.

2. Problema 2: Produtor/Consumidor com buffer limitado

2.1 Solução e decisões de projeto

As duas versões implementam a mesma interface (put, get, getTimeout, tamanho), então os produtores e consumidores são o mesmo código. **Versão com channel:** o buffer é make(chan int, K); o próprio Go bloqueia o produtor com o buffer cheio e o consumidor com o buffer vazio. **Versão com semáforos:** o buffer é um slice circular de K posições com dois semáforos de contagem, notFull (começa em K) e notEmpty (começa em 0), e dois mutexes. O semáforo é implementado com um channel de tokens (P = receber, V = enviar), pois a biblioteca padrão não tem semáforo. O algoritmo é: put: P(notFull); Lock(mEntra); itens[entra]=x; entra++; Unlock(mEntra); V(notEmpty) e get: P(notEmpty); Lock(mSai); x=itens[sai]; sai++; Unlock(mSai); V(notFull). Com vários produtores e consumidores os semáforos sozinhos não bastam: sem mEntra dois produtores poderiam receber a mesma posição de escrita e um sobrescreveria o outro; sem mSai dois consumidores poderiam retirar o mesmo item. Usamos dois mutexes diferentes para que um produtor e um consumidor possam trabalhar ao mesmo tempo. O semáforo é obtido antes do mutex para ninguém bloquear esperando vaga/item enquanto segura o mutex. Não há data race entre a escrita e a leitura da mesma posição porque o V(notEmpty) só ocorre depois da escrita (e o V(notFull) depois da leitura), o que cria a relação happens-before do modelo de memória de Go (confirmado com -race).

Consumo com timeout: o consumidor 0 usa select { case x := <-...: ...; case <-time.After(d): ... } nas duas versões. Se nenhum item chega em d, ele executa o comportamento alternativo (conta o timeout e imprime uma mensagem nas 3 primeiras vezes) e volta a tentar. **Verificação:** cada item tem um identificador único (produtor*itens + sequência); ao final o programa confere se o total consumido é igual ao produzido e se nenhum identificador foi consumido duas vezes ou perdido. A saída mostra OK ou ERRO em cada linha; em todas as execuções deste relatório foi OK.

2.2 Encerramento ordenado

Depois que todos os produtores terminam (WaitGroup), a main coloca no buffer uma **pílula de fim** (valor especial -1) para cada consumidor. Como o buffer é FIFO, as pílulas ficam atrás de todos os itens reais: um consumidor só recebe a pílula depois que os itens anteriores já foram retirados, ou seja, o buffer é drenado e nenhum item se perde. Cada consumidor encerra ao receber uma pílula, e a main espera os consumidores. É o mesmo mecanismo nas duas versões. Não há deadlock porque ninguém fica esperando um item que nunca virá: há exatamente uma pílula por consumidor. O consumidor 0 (com timeout) também sai ao receber a pílula.

2.3 Deadlock e starvation

Não ocorre deadlock em nenhuma das versões: todas as execuções terminaram, também com K=1 (o caso de mais bloqueio) e com o encerramento por pílulas. Os produtores só esperam vaga, que os consumidores liberam, e os consumidores só esperam itens, que os produtores entregam; não há dependência circular entre dois recursos e nenhum mutex é mantido durante uma espera. Starvation: as filas de espera dos channels são servidas em ordem de chegada, então ninguém é ignorado indefinidamente; no cenário equilibrado cada consumidor processou entre 1975 e 2024 dos ~2000 itens. A exceção é o consumidor 0 com timeout: a cada timeout ele volta ao fim da fila de espera e, no cenário 'consumidor rápido', recebeu só 128 a 156 itens contra ~615 dos outros. Isso é efeito do timeout, não do buffer.

2.4 Efeito da capacidade K

Cenário (P=C=4)	Versão	K	Itens/s	Ocupação média	Espera média no put	Itens por consumidor (C0 C1 C2 C3)
Equilibrado (prod=cons=0,5 ms, 2000 itens/prod.)	channel	1	4443	0,52	59 µs	2017 1994 2006 1983
	channel	10	4146	4,61	9 µs	1975 2015 2008 2002
	channel	100	4301	24,85	0	1988 2024 1979 2009
	semáforos	1	4127	0,50	62 µs	2020 1978 2010 1992
	semáforos	10	4413	4,37	6 µs	2002 2009 2006 1983
	semáforos	100	4336	36,28	0	2010 2000 2005 1985
Sem trabalho (só sincronização, 50000 itens/prod.)	channel	1	2.289.673	0,59	2 µs	30599 58182 55907 55312
	channel	10	3.681.397	5,48	1 µs	23851 57451 59821 58877
	channel	100	4.912.315	46,34	1 µs	20113 57313 60421 62153
	semáforos	1	1.401.108	0,00	3 µs	48867 50430 50456 50247
	semáforos	10	1.844.391	2,73	2 µs	40613 54750 53000 51637
	semáforos	100	2.556.515	55,61	2 µs	43396 53321 50780 52503
Produtor rápido (prod=0, cons=1 ms)	channel	1	2981	1,00	1,34 ms	2022 2003 1988 1987
	channel	10	2779	9,99	1,44 ms	1997 1975 2046 1982
	channel	100	2837	99,46	1,37 ms	2013 1974 1999 2014
	semáforos	1	2905	0,99	1,38 ms	1974 2019 1975 2032
	semáforos	10	2859	9,98	1,40 ms	2002 2008 2013 1977
	semáforos	100	2830	99,36	1,38 ms	1984 2024 1983 2009
Consumidor rápido (prod=3 ms, cons=0, timeout 2 ms, 500 itens/prod.)	channel	1	1170	0,00	0	155 615 614 616 (623 timeouts)
	channel	10	1154	0,00	0	128 624 624 624 (632)
	channel	100	1144	0,00	0	145 619 617 619 (624)
	semáforos	1	1176	0,00	0	146 618 618 618 (621)
	semáforos	10	1120	0,00	0	148 617 617 618 (643)
	semáforos	100	1130	0,00	0	156 615 614 615 (621)

Uma execução por configuração. Ocupação = média de amostras do tamanho do buffer a cada 200 µs. Itens/s = itens consumidos / tempo total. Espera no put = tempo médio que o produtor ficou bloqueado em cada put.

Efeito de K e desacoplamento. K é a folga que deixa produtores e consumidores terem ritmos diferentes sem bloquear um ao outro. No cenário **equilibrado**, a vazão fica em ~4100-4400 itens/s para qualquer K (a diferença entre K=1 e K=100 é ruído: a maior vazão foi justamente a de K=1 no channel), porque o limite é o tempo de trabalho simulado. O efeito de K aparece no bloqueio: com K=1 o produtor espera ~60 µs por item (quase um encontro entre produtor e consumidor), com K=10 cai para 6-9 µs e com K=100 para ~0, enquanto a ocupação média sobe de 0,5 para ~4,5 e para 25-36: o buffer absorve a variação aleatória dos tempos. No **produtor rápido** o buffer fica sempre cheio (ocupação ≈ K) e o produtor bloqueia ~1,4 ms por item em qualquer K: quem manda é o consumidor (~2800-3000 itens/s), e um K maior só guarda mais itens esperando. No **consumidor rápido** acontece o oposto: o buffer fica vazio (ocupação 0), os consumidores é que esperam e K é irrelevante; nesse cenário o consumidor 0 estoura o timeout ~620-640 vezes e executa a tarefa alternativa. No cenário **sem trabalho**, em que só se mede o custo da sincronização, K tem grande efeito: no channel a vazão sobe de 2,3 para 4,9 milhões de itens/s (2,1x) de K=1 para K=100, e nos semáforos de 1,4 para 2,6 milhões (1,8x), porque com mais folga há menos bloqueios e trocas de contexto entre goroutines. Ou seja, o desacoplamento só vira ganho de vazão quando a sincronização é o gargalo.

Channel x semáforos. Com trabalho simulado as duas versões são indistinguíveis dentro do ruído. Sem trabalho, o channel foi de 1,6x a 2,0x mais rápido: cada item custa uma operação de channel, que o runtime do Go já implementa com um buffer circular e um lock interno, contra duas operações de semáforo (P e V, cada uma um channel) mais um mutex na versão manual. A versão com semáforos tem o mesmo comportamento de bloqueio, mas com mais trabalho de sincronização por item. Observação: nesse cenário o consumidor 0 consumiu bem menos itens (20-31 mil contra ~57 mil no channel), o que atribuímos (hipótese) ao custo de criar um timer a cada chamada de `time.After`, e não a injustiça do buffer. Como cada configuração foi medida uma vez, diferenças pequenas entre linhas não devem ser interpretadas.

3. Uso de ferramentas de IA

Foi utilizado o **Claude Code (modelo Claude Sonnet 5.5, da Anthropic)**. Etapas: **projeto da solução** (escolha das estratégias, do protocolo de encerramento e da interpretação de R); **geração de código** (os dois programas em Go e os scripts de coleta); **depuração e validação** (instalação do Go e do gcc de 64 bits, execução com `-race`, e uma simplificação do código a pedido dos alunos para facilitar a compreensão e a apresentação); **análise dos dados** (leitura das saídas e rascunho das interpretações); **redação** do relatório e do README. Os experimentos foram realmente executados e todos os números deste relatório vêm das saídas guardadas em `relatorio/dados/`. As explicações marcadas como hipótese não foram verificadas por experimento próprio. Cada integrante deve ser capaz de explicar qualquer parte do código e das análises.

4. Como compilar e executar

Go >= 1.22 (para `-race` no Windows é preciso um gcc de 64 bits no PATH). `cd filosofos; go run -race . -n 5 -r 1000 -estrategia todas | cd produtor-consumidor; go run -race . -p 4 -c 4 -k 1,10,100 -versao ambas | -h` lista todas as opções. Mais detalhes no README.